

Week 2 Glossary: Docker 실행 환경 표준화

이 용어집은 Docker 명령어 암기보다 운영 질문을 먼저 잡기 위한 문서다. 새 용어를 볼 때는 "이것이 image를 만드는 문제인가, container를 실행하는 문제인가, network/storage/config를 연결하는 문제인가"로 분류한다.

Term	뜻	운영 관점 질문
Docker	애플리케이션 실행 환경을 image와 container로 표준화하는 도구와 생태계	같은 실행 조건을 다른 사람이 재현할 수 있는가
Docker Desktop	로컬 장비에서 Docker를 쉽게 실행하고 관리하게 해주는 데스크톱 앱	macOS/Linux에서 설치 방식, 권한, 실행 상태가 맞는가
Docker Engine	image build, container run, network, volume을 실제로 처리하는 Docker 실행 엔진	CLI 명령이 어느 daemon에 전달되는가
Docker CLI	terminal에서 docker ... 명령을 실행하는 client	CLI는 있는데 daemon 연결이 안 되는 상태를 구분했는가
Docker daemon	Docker Engine의 background service	docker version에서 Server 정보가 보이는가
Docker context	CLI가 어느 Docker endpoint를 대상으로 명령을 보내는지 정하는 설정	엉뚱한 daemon이나 원격 context를 보고 있지 않은가
Image	container를 만들기 위한 읽기 중심 실행 패키지	runtime, app code, config default가 무엇을 포함하는가
Container	image에서 시작된 실행 중인 process와 격리된 실행 환경	현재 running/stopped/exited 상태인가
Container lifecycle	create, start, stop, remove로 이어지는 container 상태 흐름	실행 후 정리까지 README에 남겼는가
Registry	image를 저장하고 내려받는 저장소	image 출처를 신뢰할 수 있는가
Docker Hub	Docker의 대표 public registry	public image 사용 시 tag와 출처를 확인했는가
Dockerfile	image를 만들기 위한 instruction 파일	빌드 절차가 문서화되어 재현 가능한가
Instruction	Dockerfile의 FROM, COPY, RUN, CMD 같은 한 줄 명령	각 instruction이 image에 무엇을 추가하는가
FROM	base image를 고르는 instruction	출처와 tag를 설명할 수 있는가
WORKDIR	이후 instruction과 container 시작 위치를 정하는 instruction	상대 경로가 어디 기준인지 설명할 수 있는가
COPY	build context의 파일을 image 안으로 복사하는 instruction	secret이나 불필요한 파일이 복사되지 않는가
RUN	image build 시점에 실행되는 instruction	runtime 명령과 build-time 명령을 구분했는가
CMD	container 시작 시 기본으로 실행할 명령	container의 주 process가 무엇인가
Build context	docker build가 image로 보낼 수 있는 파일 범위	불필요한 파일이나 secret이 포함되지 않았는가
.dockerignore	build context에서 제외할 파일 목록	.env, .git, 큰 임시 파일을 제외했는가

Layer	image를 구성하는 filesystem 변경 단위	cache가 어디서 재사용되고 어디서 깨지는가
Build cache	이전 build layer를 재사용해 build를 빠르게 하는 기능	변경한 파일이 cache 때문에 반영되지 않았다고 오해하지 않는가
Tag	image 이름에 붙이는 사람이 읽기 쉬운 version label	latest만 쓰지 않고 의미 있는 tag를 남겼는가
Digest	image 내용을 기준으로 한 고유 식별값	같은 tag가 다른 내용을 가리킬 위험을 어떻게 줄이는가
Pull	registry에서 image를 내려받는 작업	어떤 image:tag를 받았는가
Push	local image를 registry에 올리는 작업	public push 전에 secret과 license를 확인했는가
Port binding	host port를 container port에 연결하는 설정	사용자가 어떤 port로 접속하고 충돌은 없는가
Host port	내 컴퓨터에서 접속하는 port	localhost:18080의 18080이 비어 있는가
Container port	container 내부 process가 듣는 port	nginx는 보통 container port 80을 사용한다
Volume	container lifecycle 밖에 데이터를 보존하는 저장 영역	container 삭제 후 데이터가 남아야 하는가
Bind mount	host directory/file을 container에 연결하는 방식	개발 중 파일 변경을 바로 반영해야 하는가
Named volume	Docker가 이름으로 관리하는 volume	DB 데이터처럼 장기 보존할 데이터인가
Environment variable	실행 시점에 주입하는 설정 값	설정을 image에 굳히지 않고 바꿀 수 있는가
Secret	password, token, key처럼 노출되면 안 되는 값	값이 repository, image layer, screenshot에 남지 않았는가
Bridge network	Docker container들이 기본적으로 연결되는 가상 network 방식	container끼리 어떻게 이름과 port로 통신하는가
DNS service name	Compose network에서 service 이름으로 서로 찾는 방식	app이 localhost가 아니라 db 같은 service name을 쓰는가
Compose	여러 container를 하나의 YAML 파일로 정의하고 실행하는 Docker 도구	명령어 묶음을 파일로 재현 가능하게 관리하는가
Service	Compose에서 하나의 container 실행 단위를 정의하는 항목	app, db, cache 같은 역할이 명확한가
compose.yaml	Compose application model을 담은 YAML 파일	services, ports, environment, volumes, networks가 설명 가능한가

depends_on	Compose service 시작 순서를 표현하는 설정	시작 순서와 실제 readiness를 혼동하지 않는가
Healthcheck	container 내부 상태를 주기적으로 확인하는 설정	process running과 service ready를 구분했는가
Restart policy	container 종료 후 재시작 방식을 정하는 설정	실습에서는 예기치 않은 재시작이 evidence를 흐리지 않는가
docker logs	container의 stdout/stderr를 확인하는 명령	장애 증거가 log에 남아 있는가
docker exec	실행 중인 container 안에서 명령을 실행하는 방법	내부 상태를 확인해야 하는 이유가 있는가
docker inspect	container/image/network/volume의 상세 JSON 상태를 보는 명령	추측 대신 실제 설정을 확인했는가
docker ps	container 목록과 상태를 보는 명령	실행 중인지, port가 무엇인지 확인했는가
docker rm	stopped container를 삭제하는 명령	container와 image 삭제를 구분했는가
docker rmi	image를 삭제하는 명령	사용 중인 container가 남아 있지 않은가
Cleanup	사용하지 않는 container, image, volume을 정리하는 작업	disk 낭비와 잘못된 재실행을 줄였는가

초보자 혼동 방지 표

헷갈리는 쌍	구분
Image vs Container	image는 실행 재료, container는 실행 중이거나 실행됐던 instance
Build vs Run	build는 image 생성, run은 container 실행
Host port vs Container port	host port는 내 브라우저가 접속하는 문, container port는 앱이 내부에서 듣는 문
Bind mount vs Named volume	bind mount는 host 경로가 보이고, named volume은 Docker가 이름으로 관리한다
Environment variable vs Secret	env는 주입 방식이고 secret은 보호가 필요한 값의 성격이다
Docker Desktop vs Docker Engine	Desktop은 GUI/통합 환경, Engine은 실제 container 작업을 처리하는 실행 계층이다
Compose service vs Container	service는 설계 정의, container는 그 정의로 만들어진 실행 instance다
Running vs Ready	process가 떠 있어도 HTTP/DB가 받을 준비가 안 됐을 수 있다

설치/권한 오류 용어

증상/용어	뜻	먼저 볼 것
command not found: docker	CLI가 설치되지 않았거나 PATH에서 찾지 못함	Docker Desktop/Engine 설치, terminal 재시작
Cannot connect to the Docker daemon	CLI는 있으나 daemon/Server에 연결 실패	Docker Desktop running, systemctl status docker
Permission denied	Linux 사용자가 Docker socket 접근 권한이 없을 수 있음	sudo docker ... 성공 여부, docker group 정책
Port already allocated	host port를 이미 다른 process/container가 사용 중	docker ps, 다른 port 사용
Pull access denied	image 이름, registry 권한, 로그인 문제	image name/tag, Docker Hub login
No space left on device	image/layer/volume이 disk를 많이 사용	cleanup, docker system df

Week 1에서 이어지는 말

Week 1 용어	Week 2 Docker 표현
Process	Container의 주 process
File path	Build context, bind mount source
HTTP status	curl로 container 응답 확인
Configuration	-e, .env, Compose environment
Secret	.dockerignore, runtime injection, screenshot masking
README evidence	build/run/check/stop/cleanup 절차
RCA	port conflict, env missing, volume persistence 분석